

```

# Establecer semilla aleatoria para reproducibilidad
set.seed(sample(1:100000, 1))

# --- Aleatorización de Parámetros ---
coeficientes_posibles <- c(20, 25, 30, 40, 50, 60, 75, 80, 100, 120, 150, 200)
coef_inicial <- sample(coeficientes_posibles, 1)

bases_posibles <- c(1.5, 2, 2.2, 2.5, 3)
base_exponencial <- sample(bases_posibles, 1)

factores_tiempo_posibles <- c(0.2, 0.25, 0.4, 0.5, 0.75, 0.8, 1, 1.2, 1.5)
factor_tiempo <- sample(factores_tiempo_posibles, 1)

organismos_plural <- c("ranas", "peces", "insectos", "aves", "conejos", "bacterias", "algas", "hongos",
idx_organismo <- sample(1:length(organismos_plural), 1)
organismo_plural <- organismos_plural[idx_organismo]

unidades_tiempo_plural <- c("semanas", "días", "meses", "horas", "años", "minutos")
idx_unidad <- sample(1:length(unidades_tiempo_plural), 1)
unidad_tiempo_plural <- unidades_tiempo_plural[idx_unidad]

verbos_accion <- c("se reproduce", "crece", "aumenta su población", "se multiplica", "evoluciona su can
verbo_accion <- sample(verbos_accion, 1)

contextos_estudio <- c("un estudio científico", "una observación de campo", "un experimento de laborator
contexto_estudio <- sample(contextos_estudio, 1)

simbolo_cantidad <- sample(c("r", "P", "N", "Q", "C"), 1)
simbolo_tiempo <- sample(c("t", "x", "k"), 1)

adjetivos_especie <- c("cierta", "una determinada", "una particular", "una nueva", "una exótica")
adjetivo_especie <- sample(adjetivos_especie, 1)

# --- Formulación de la función ---
funcion_texto <- glue::glue("{simbolo_cantidad}{simbolo_tiempo}) = {coef_inicial} \\cdot ({base_exponen
funcion_display <- glue::glue("{simbolo_cantidad}{simbolo_tiempo}) = {coef_inicial}({base_exponencial}

# --- Cálculo de la Solución ---
respuesta_correcta <- coef_inicial

# --- Generación de Opciones de Respuesta ---
opciones <- numeric(4)
opciones[1] <- respuesta_correcta

distractores_candidatos <- c(
  round(coef_inicial * base_exponencial), round(coef_inicial / 2), round(coef_inicial * 1.5),
  round(coef_inicial * 0.75), round(coef_inicial + base_exponencial), round(abs(coef_inicial - base_exp
  round(coef_inicial + factor_tiempo * 10), as.integer(base_exponencial), as.integer(factor_tiempo),
  sample(c(2,5,10,15,20), 1) * sample(c(1,2),1)
)

distractores_candidatos <- round(distractores_candidatos)
distractores_candidatos <- unique(distractores_candidatos)

```

```

distractores_candidatos <- distractores_candidatos[!is.na(distractores_candidatos)]
distractores_candidatos <- distractores_candidatos[distractores_candidatos > 0]
distractores_candidatos <- distractores_candidatos[distractores_candidatos != respuesta_correcta]

if (length(distractores_candidatos) >= 3) {
  opciones[2:4] <- sample(distractores_candidatos, 3)
} else {
  if (length(distractores_candidatos) > 0) {
    opciones[2:(1 + length(distractores_candidatos))] <- distractores_candidatos
  }
  valores_usados <- na.omit(opciones)
  for (i in which(is.na(opciones))) {
    nuevo_distractor <- max(valores_usados, respuesta_correcta, 1) + sample(1:5,1) + i
    while(nuevo_distractor %in% valores_usados || nuevo_distractor <= 0) {
      nuevo_distractor <- nuevo_distractor + sample(1:3,1)
      if(nuevo_distractor <=0) nuevo_distractor <- max(valores_usados, respuesta_correcta, 1) + 7
    }
    opciones[i] <- round(nuevo_distractor)
    valores_usados <- c(valores_usados, opciones[i])
  }
}
opciones <- round(as.numeric(opciones))

if(any(is.na(opciones)) || length(unique(opciones)) < 4) {
  warning("Fallback extremo activado para la generación de opciones.")
  opciones_nuevas <- c(respuesta_correcta)

  # Intentar añadir otros valores únicos de las opciones pre-existentes
  if (exists("opciones") && length(opciones) > 0) {
    for (val_existente in unique(na.omit(round(as.numeric(opciones))))) {
      if (length(opciones_nuevas) < 4 && !(val_existente %in% opciones_nuevas) && val_existente > 0) {
        opciones_nuevas <- c(opciones_nuevas, val_existente)
      }
    }
  }

  iter_seguridad_fallback <- 1
  while (length(opciones_nuevas) < 4 && iter_seguridad_fallback < 25) {
    candidato_relleno <- respuesta_correcta + sample(c(-1,1),1) * sample(1:15,1) * iter_seguridad_fallback
    candidato_relleno <- round(candidato_relleno)
    if (!(candidato_relleno %in% opciones_nuevas) && candidato_relleno > 0 && candidato_relleno != respuesta_correcta) {
      opciones_nuevas <- c(opciones_nuevas, candidato_relleno)
    }
    iter_seguridad_fallback <- iter_seguridad_fallback + 1
  }

  # Si aún no hay 4, forzar valores muy simples y únicos
  if (length(unique(opciones_nuevas)) < 4) {
    opciones_nuevas <- unique(opciones_nuevas) # Asegurar unicidad de lo que hay
    relleno_simple_idx <- 1
    while(length(opciones_nuevas) < 4){
      val_simple <- respuesta_correcta + c(5, -5, 10, -10, 1, -1, 2, -2, 15, -15)[relleno_simple_idx]
      val_simple <- round(val_simple)
    }
  }
}

```

```

    if(!(val_simple %in% opciones_nuevas) && val_simple > 0){
      opciones_nuevas <- c(opciones_nuevas, val_simple)
    } else { # Si sigue colisionando o es <=0, probar otro offset
      val_simple <- max(opciones_nuevas, respuesta_correcta, 1) + 3 + relleno_simple_idx * 2
      if(!(val_simple %in% opciones_nuevas) && val_simple > 0) {
        opciones_nuevas <- c(opciones_nuevas, round(val_simple))
      } else { # Último recurso para este slot
        opciones_nuevas <- c(opciones_nuevas, round(max(opciones_nuevas, respuesta_correcta, 1)
      )
    }
  }
  opciones_nuevas <- unique(opciones_nuevas[opciones_nuevas > 0]) # Limpiar y asegurar positivo
  relleno_simple_idx <- relleno_simple_idx + 1
  if(relleno_simple_idx > 20) break # Evitar bucle infinito extremo
}
}
opciones <- opciones_nuevas
# Asegurar que 'opciones' tenga exactamente 4 elementos únicos.
# Si después de todo el esfuerzo hay menos de 4, es un problema, pero intentamos rellenar.
if(length(unique(opciones)) < 4){
  opciones_final_final <- unique(opciones[opciones>0])
  idx_ff = 1
  while(length(opciones_final_final) < 4 && idx_ff < 10){
    val_ff = max(opciones_final_final, respuesta_correcta, 1) + idx_ff * sample(1:3,1) + sample(1:3,1)
    if(!val_ff %in% opciones_final_final && val_ff > 0) opciones_final_final <- c(opciones_final_final, val_ff)
    idx_ff = idx_ff + 1
  }
  # Si todavía no, forzar secuencia simple
  if(length(unique(opciones_final_final)) < 4) {
    opciones_final_final <- c(respuesta_correcta, respuesta_correcta+1, respuesta_correcta+2, respuesta_correcta+3)
    opciones_final_final <- unique(round(opciones_final_final[opciones_final_final>0]))
    # Rellenar si respuesta_correcta es negativa o algo raro pasó
    idx_fff = 1
    while(length(opciones_final_final) < 4 && idx_fff < 10){
      opciones_final_final <- c(opciones_final_final, max(opciones_final_final,1)+idx_fff)
      opciones_final_final <- unique(round(opciones_final_final[opciones_final_final>0]))
      idx_fff = idx_fff + 1
    }
  }
  opciones <- opciones_final_final[1:4] # Tomar los primeros 4, puede haber NA si no se generaron 4
}
# Asegurar que la respuesta correcta esté en las opciones finales
if (!respuesta_correcta %in% opciones) {
  # Buscar un NA para reemplazar, si no, uno aleatorio
  idx_replace_rc <- if(any(is.na(opciones))) which(is.na(opciones))[1] else sample(1:4,1)
  opciones[idx_replace_rc] <- respuesta_correcta
}
# Última limpieza de NAs y duplicados si la RC causó uno
if(any(is.na(opciones)) || length(unique(opciones)) < 4){
  opciones_limpias <- unique(na.omit(opciones))
  idx_fill_last = 1
  while(length(opciones_limpias) < 4 && idx_fill_last < 10){
    val_fill_last = max(opciones_limpias, respuesta_correcta, 1) + idx_fill_last * 3 + sample(1:3,1)
    if(!val_fill_last %in% opciones_limpias && val_fill_last > 0) opciones_limpias <- c(opciones_limpias, val_fill_last)
    idx_fill_last = idx_fill_last + 1
  }
}

```

```

      idx_fill_last = idx_fill_last + 1
    }
    if(length(opciones_limpias) >= 4) opciones <- opciones_limpias[1:4]
    else { # Fallback absoluto
      opciones <- round(c(respuesta_correcta, respuesta_correcta+sample(1:5,1), respuesta_correcta+
      opciones <- unique(opciones)
      # Si aún no son 4 únicos (ej. por redondeo o RC muy grande), ajustar
      add_val = 1
      while(length(opciones) < 4){
        opciones <- unique(c(opciones, respuesta_correcta + 15 + add_val))
        add_val = add_val + 1
      }
    }
  }
}

opciones_ordenadas <- round(opciones) # Asegurar enteros al final
solucion_logica <- (opciones_ordenadas == respuesta_correcta)

if(any(is.na(opciones_ordenadas))) { stop("CRÍTICO: 'opciones_ordenadas' contiene NAs.") }
if(length(unique(opciones_ordenadas)) < 4) { stop("CRÍTICO: 'opciones_ordenadas' no tiene 4 opciones únicas.") }
if(sum(solucion_logica) != 1) {
  stop(paste0("CRÍTICO: 'solucion_logica' no tiene un TRUE. Sol: ", paste(solucion_logica, collapse="
  ". Opt: ", paste(opciones_ordenadas, collapse=","), ". RC: ", respuesta_correcta))
}

```

Question

una exótica especie de conejos aumenta su población según la función $C(x) = 40 \cdot (2.5)^{0.75 \cdot x}$, donde x se mide en minutos y C es la cantidad de conejos.

De acuerdo con la información anterior, la cantidad de conejos que había al iniciar un análisis poblacional, es:

Answerlist

- 40 conejos.
- 42 conejos.
- 48 conejos.
- 2 conejos.

Solution

Para determinar la cantidad de conejos que había al iniciar un análisis poblacional, necesitamos evaluar la función de población $C(x) = 40(2.5)^{0.75x}$ en el momento inicial, es decir, cuando el tiempo $x = 0$.

La función dada es: $C(x) = 40(2.5)^{0.75x}$

Sustituimos $x = 0$ en la función: $C(0) = 40(2.5)^{0.75 \cdot 0}$ $C(0) = 40(2.5)^0$

Cualquier número (distinto de cero) elevado a la potencia 0 es igual a 1. Por lo tanto, $(2.5)^0 = 1$. $C(0) = 40 \cdot 1$
 $C(0) = 40$

Así, la cantidad de conejos que había al iniciar un análisis poblacional es 40.

Answerlist

- Verdadero
- Falso
- Falso
- Falso

Meta-information

exname: crecimiento_exponencial_valor_inicial extype: schoice exsolution: 1000 exshuffle: TRUE exsection: Álgebra y Cálculo/Funciones Exponenciales/Modelado de Poblaciones extol: 0.01 expoints: 1